

1. Introduction

Our project is a Spotify Controller that enables users to connect their Spotify Web Application account to an Arduino and breadboard setup. Through this integration, users can interact with the circuit to control music playback on Spotify seamlessly.

The program assumes the user has access to the Spotify Web Application and a valid access token. Once a song is playing, users can engage with the circuit using the following features:

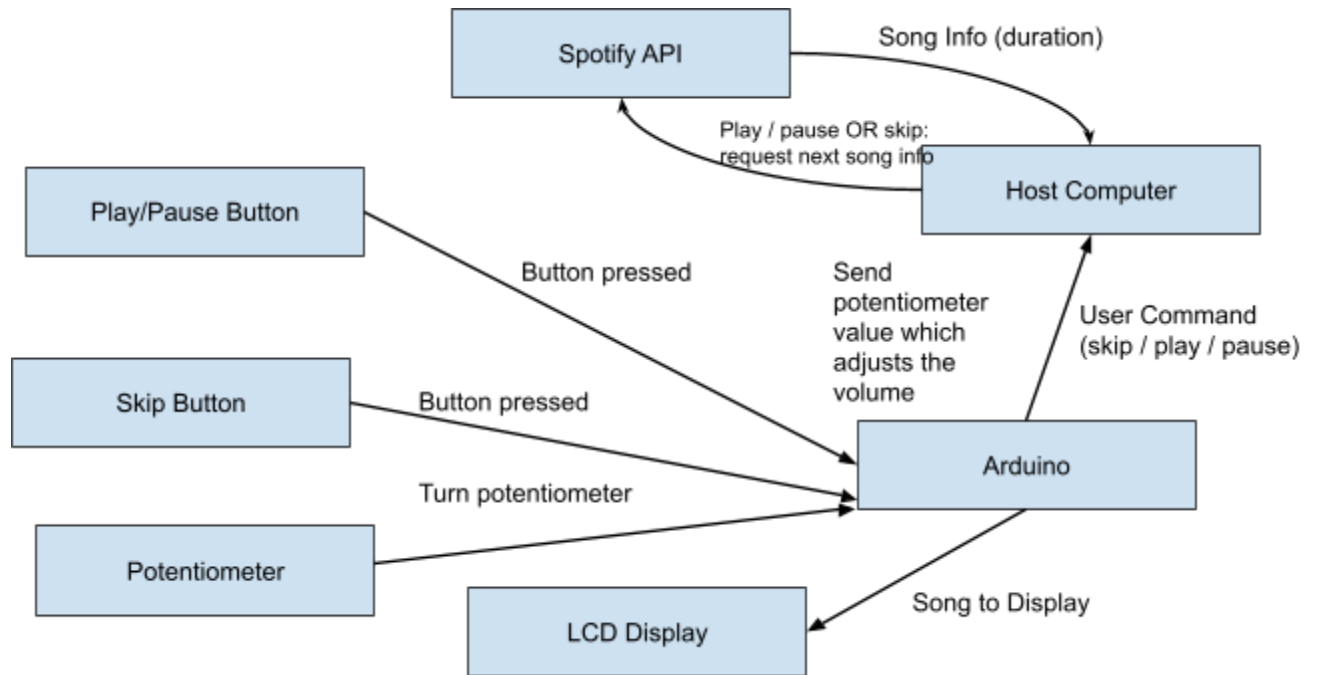
- **Play/Pause:** Pressing the inner button toggles playback of the current song.
- **Skip Track:** Pressing the outer button skips the current song and plays the next one in the playlist or queue.
- **Volume Control:** Turning the potentiometer adjusts the volume up or down of the music on the host computer.

Additionally, the setup includes an LCD display that shows the title of the current song and a progress bar indicating the song's elapsed duration. Users can freely interact with the buttons and potentiometer in any order, allowing dynamic control of their Spotify playlist.

2. Revised Requirements

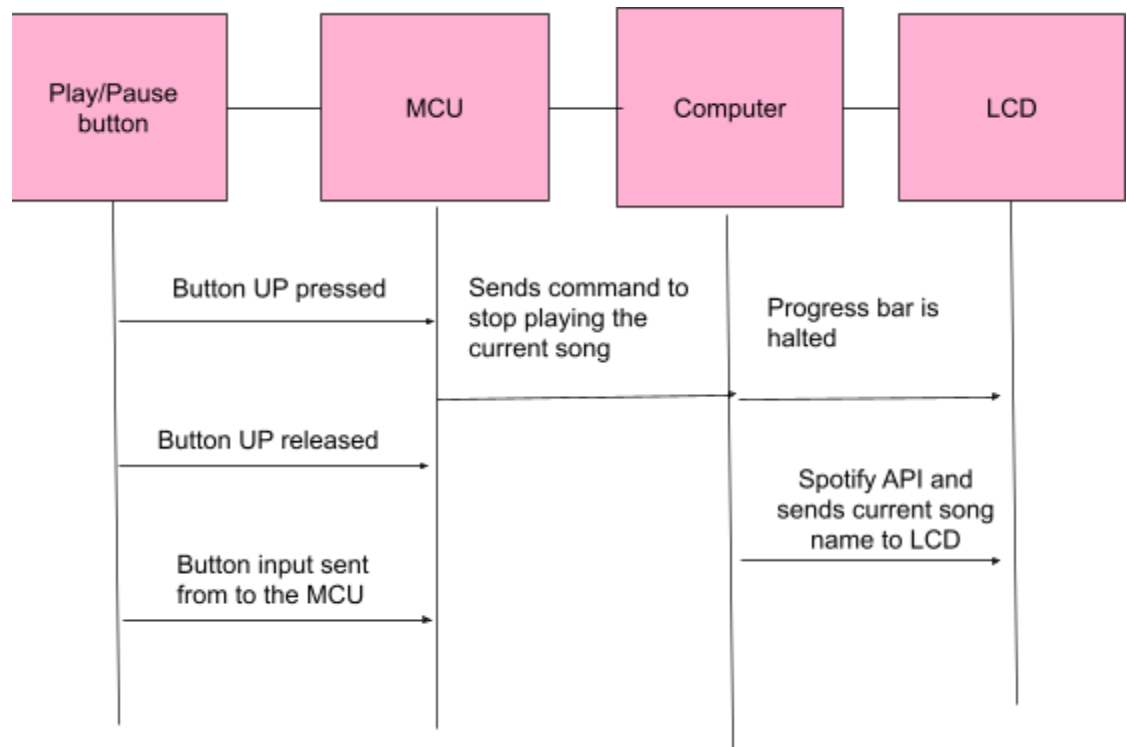
- a) Shall communicate with the powering device using Serial to execute Spotify commands
- b) Shall use HTTP requests using the Spotify API to communicate with Spotify
- c) Shall show current playback of song on physical display
- d) Shall show currently playing song on physical display
- e) Shall skip to the next song when the physical skip button is pressed
- f) Shall begin playing a song when the physical play button is pressed if there is a current song or if the song is paused
- g) Shall stop playing a song when the physical pause button is pressed if there is a current song and the song is playing
- h) Shall change the controlling computer's volume based on potentiometer value

3. Revised Architecture Diagram



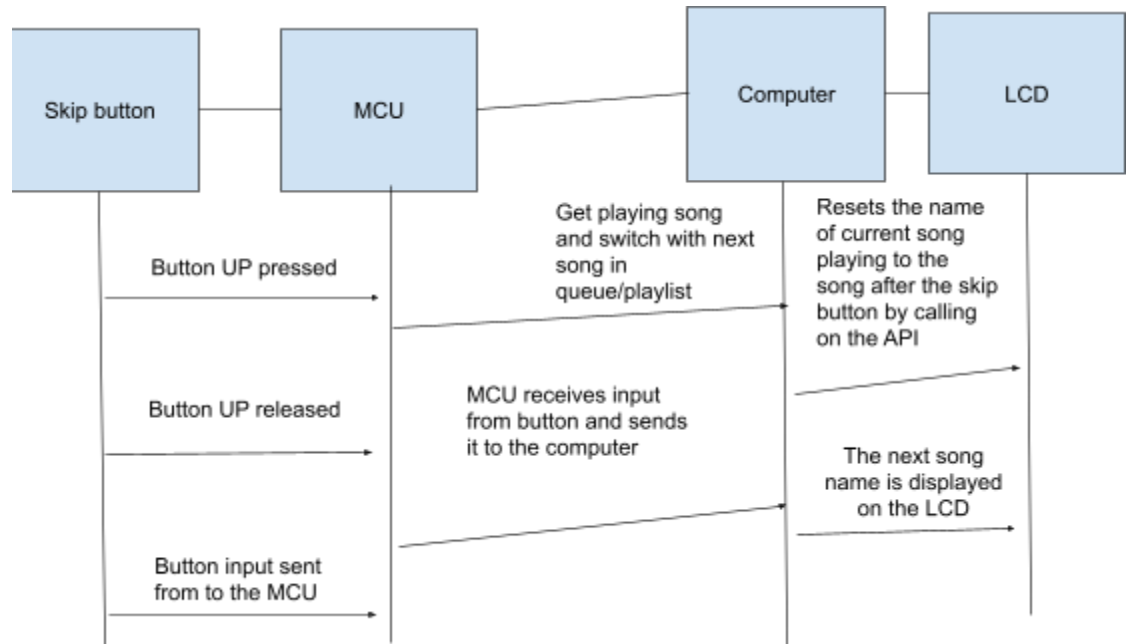
4. Three Sequence Diagrams

1. In this scenario, a user is playing a current song on the spotify app on their computer and want to use the pause button to stop the song so they press button and the microcontroller communicates with the computer which is running the spotify api to stop the song which is then reflected on stopped progression of the progress bar on the LCD.

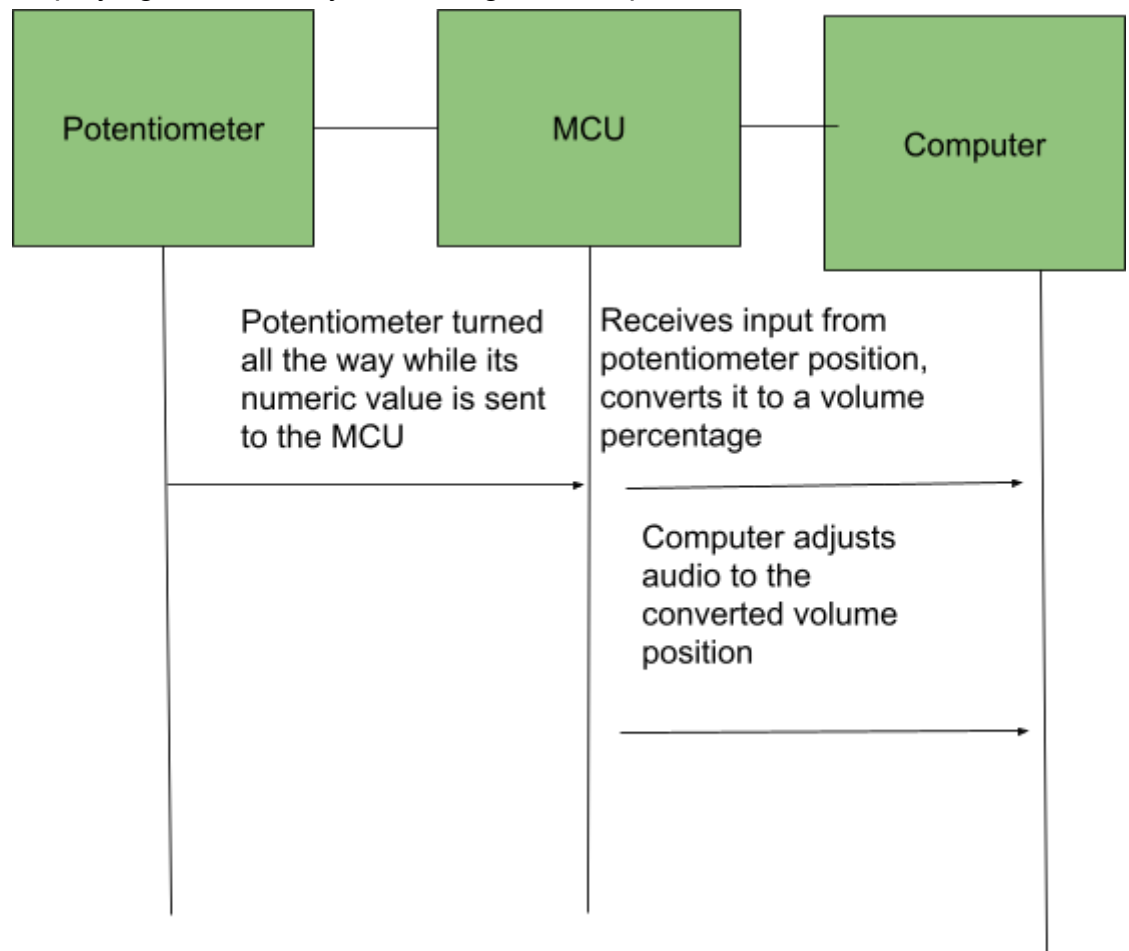


a)

2. In this scenario, the user engages the skip button which sends a request to the spotify application to play the next song in the queue and the LCD adjusts accordingly.



3. In this scenario, a user can either adjust the volume up or down of the host computer playing the music by interacting with the potentiometer.



5. Revised Finite and/or Extended State Machine

States:

1. wait_for_song
2. paused
3. Playing

Our FSM detects user input through the playFlag and skipFlag due to the play and skip buttons working through interrupts. The interrupts are attached to the corresponding pins, and when the interrupt is triggered, it then sets the appropriate flag, which the FSM checks for. This is how our inputs are communicated to our FSM.

Interrupt Variables

playFlag	<i>Whether the play/pause button has been pressed since last loop</i>	False
skipFlag	<i>Whether the skip button has been pressed since last loop</i>	False

Spotify API Local Variables

Variable name	Description	Initial Value
name	<i>The "name" field in the JSON we received from the python code</i>	N/A
duration	<i>The "duration" field in the JSON we received from the python code</i>	N/A
progress	<i>The "progress" field in the JSON we received from the python code</i>	N/A

Global Variables (used by fsm, but not direct inputs)

Variable name	Description	Initial Value
---------------	-------------	---------------

songDuration	<i>Total length of current song in ms</i>	0
songName	<i>The name of the song that is currently loaded</i>	"Example Song"
songSyncTime	<i>The progress through the song upon loading new song from one song finishing</i>	0

Inputs

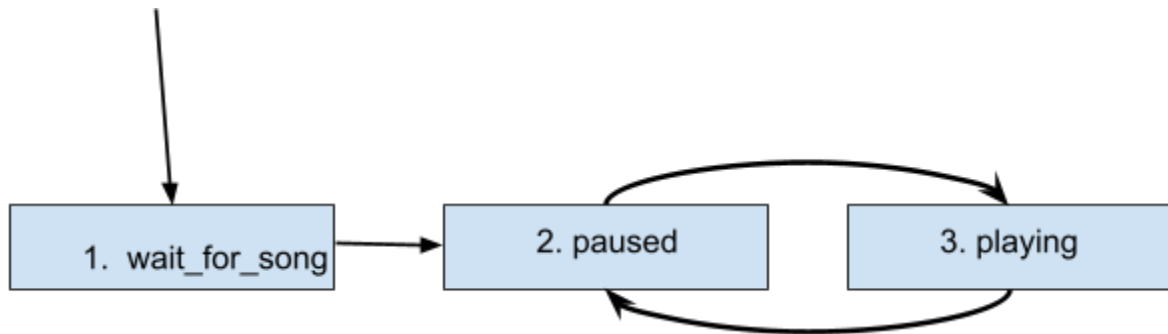
- The only input in our FSM is curState – the current state
- We handle input from the Spotify API separately from the FSM. We check every loop if any new song information has been sent from the python code, and we immediately parse the JSON and set the global variables if so.
- We handle potentiometer input also in loop() using analogRead, since this input is not state-dependent. We need to be able to adjust volume in any state.

FSM Outputs

- Serial.println("skip") – signal to the python code that we want to skip to the next song
- Serial.println("play") – signal to the python code that we want to play the song
- Serial.println("pause") – signal to the python code that we want to pause the song
- songTimer – stopwatch class that tracks song progress
 - reset() – reset the stopwatch
 - start() – start the stopwatch
 - stop() – stop the stopwatch
- displayProgressBar – clears the progress bar on the LCD (just dashes)
- updateProgressBar – display the progress of the song in bars on the LCD

General Outputs

- displaySongName() – displays a new song name onto the second row of the LCD
- setTimeBetweenUpdate() – update the variable tracking how much time each bar in the progress bar takes



Transition	Guard	Explanation	Output	Variables
1-2	songName != "Example Song"	Arduino receives initial song info from laptop after startup		
2-3 (a)	skipFlag == True	Skip button is pressed while paused	songTimer.reset() Serial.println("skip")	songName = name songDuration = duration skipFlag = false
2-3 (b)	playFlag == True	Play button is pressed while paused	songTimer.start()	playFlag = false
3-2	playFlag == True	Pause button is pressed while playing	songTimer.stop()	playFlag = false
3-3 (a)	skipFlag == True	Skip button is pressed while playing	Serial.println("skip")	skipFlag = false
3-3 (b)	song_progress >= (songDuration + 1000)	Song ends based on our local timer, with some buffer time	Serial.println("getSongInfo");	songName = name songDuration = duration songSyncTime = progress

6. Revised Traceability Matrix

Transiti on	R1	R2	R3	R4	R5	R6	R7	R8
1-2	x	x		x				N/A
2-3 (a)	x	x	x	x	x			N/A
2-3 (b)				x		x		N/A
3-2				x			x	N/A
3-3 (a)	x	x	x	x	x			N/A
3-3 (b)	x	x	x	x				N/A

7. Testing Approach

Unit Testing

- a) Spotify API interface: mock API requests for play, pause, skip, and song info to verify that the system can properly send commands and handle responses
- b) Buttons: simulate button presses and verify signals to the Arduino/MCU to confirm that the physical buttons correctly trigger play/pause/skip commands
- c) LCD display: display mock playback data on the LCD and verify that the updates are correct to verify that the display shows the progress accurately

Integration Testing

- a) Buttons → Arduino/MCU → Host Computer: verify that pressing the buttons results in correct commands being sent from the MCU to the computer to confirm the communication pathway is working
- b) Host Computer → Spotify API → Host Computer: test end-to-end playback control (play/pause/skip) by simulating button presses to make sure that commands reach the API and return the expected playback status
- c) Host Computer → Arduino/MCU → Display: update playback information from the API and confirm that it appears on the display to verify that the data is accurately displayed

System Testing

- a) Playback control: real user actions to update playback status
- b) Power and connectivity: scenario of us powering on the system and confirming that the components connect properly

Our testing approaches will use automated testing unit tests for the software portion (Spotify API and host computer). We will also use mocking to simulate API responses for testing. We will also be using hardware testing since physical components are involved. Lastly, we will be using scenario-based testing to simulate user actions and verify that we're getting appropriate responses from the system in our controlled environment. We know that our testing will be enough when all tests are executed without failure, that we get consistent results after running the tests multiple times, and if it performs as expected in real-world conditions.

8. Safety and Liveness Requirements

1. (Safety) $G(\text{Serial.println}(\text{"play"}) \Rightarrow \text{playFlag}) \vee G(\text{Serial.println}(\text{"pause"}) \Rightarrow \neg \text{playFlag})$

Sending a request for play or pause always leads to the flag being toggled.

2. (Safety) $G(50\text{ms} \leq (\text{curTime} - \text{lastVolumeChangeTime}) \leq 200\text{ms})$

When changing the volume, when we are sending messages to change the volume, send requests at intervals between 50 and 200 milliseconds, so the system will not be overwhelmed.

3. (Liveness) $F(\text{httpResponse} \neq \text{null})$ // TODO

For each HTTP request sent, we require the response to come in eventually, and stop being null.

4. (Liveness) $\text{songName} \neq \text{prevSongName} \Rightarrow F(\text{displayName} = \text{songName})$

When a song changes and the song name becomes different, we want the display to eventually update.

5. (Liveness) $\text{song_progress} \geq \text{songDuration} \Rightarrow F(\text{songName} \neq \text{prevSongName})$

When the last song ends, we require a new song to be loaded eventually, represented by the name being different.

9. Environment Processes

There are primarily 2 environment processes here, the first being the user themselves and all their actions, and the second being Spotify's audio player. These are the two direct outside influences on the operation of the FSM.

For the user, we can use a hybrid, nondeterministic system to model everything they can do. There are only 3 ways the user can interact with the system, that being the skip button, the play/pause button, and the volume potentiometer. While both of the first two are clearly discrete, the potentiometer control is better modeled as continuous input, so we would like a hybrid system. On top of that, the user has completely arbitrary control over the system, in that they could choose to change the audio or play/skip the current song at any point they want, thus nondeterminism makes the most sense.

For the Spotify audio player, this process can be a discrete deterministic system. The Spotify audio player itself only interacts with the system when the current song has finished playing and a new song begins, indicating to the system that new song data needs to be requested and given to the user. In that sense then, the only information we need from the audio player itself is when the song ends, which is a discrete action. On top of that, the length of each song is known ahead of time and plays at a fixed rate, meaning that when songs end can be found deterministically.

10. File Descriptions

pycache

Directory to store compiled bytecode files. When we run our Python program, the Python interpreter compiles the source code into bytecode files to speed up execution.

spotify (directory)

Virtual environment for our program that handles the Python script for sound functionality in our system.

venv (directory)

Virtual environment for our program that handles the Python script for the API calls. This includes API calls to pause, play, skip, and reset songs, as well as getting the duration and name of the song.

arduino_secrets.py

File used to store sensitive information for our Python script. This file contains our client ID, client secret, the spotify market country code, and the access token that is used to authenticate API requests.

arduino_tests.py

File containing unit tests for the Python script, as well as a handy runner class that will run all of the unit tests together.

audio.py

File that is used to interact with our Arduino device and control the system volume on a computer. The `set_volume` function can be called with a volume level to adjust the system volume based on input from the Arduino (potentiometer). The `clean_data` function is used to preprocess the data received from the Arduino before using it.

io_utils.h

Header file that declares functions related to input and output operations. The file uses header guards to prevent multiple inclusions.

io_utils.ino

Arduino sketch that implements the functions declared in the `io_utils.h` file. We are able to write to the buffer (`writeToBuf`) and deserialize JSON data from the buffer into a `JsonDocument` (`readResponse`).

main.py

Main file of the project that contains a Python script to interact with the Spotify API to control playback. It handles various HTTP response status codes and manages authorization by checking for a valid refreshToken. It also processes messages from the Arduino and calls appropriate handler functions.

requirements.txt

File used in our Python script to specify the dependencies required for the project to run. It includes pyserial for serial communication, requests for making HTTP requests, and osascript for running AppleScript commands on macOS.

SpotifyController.h

Header file for the Arduino that interacts with the Spotify API. It defines constants and includes necessary libraries for Arduino programming, JSON handling, and Watchdog Timer functionality.

SpotifyController.ino

Arduino sketch that interacts with the Spotify API and uses a Watchdog Timer to ensure the system remains responsive. It refreshes the timer periodically to prevent the Arduino from resetting. The code reads data from the serial input and stores it in a buffer, and if data has been written to the buffer, it processes the data by parsing it into a JsonDocument.

SpotifyUtils.h

Header file for our Arduino that provides utility functions related to setting up utilities, updating inputs, displaying the song name, displaying and updating a progress bar, and clearing the song name.

SpotifyUtils.ino

Arduino sketch that provides implementations for the utility functions declared in SpotifyUtils.h. It includes the LiquidCrystal library for controlling an LCD and the SpotifyUtils.h header for utility function declarations. It defines the pin numbers for the play/pause and skip buttons and initializes variables to store the states of the button.

SongTimer.h

Header file to track and manage time for a song playback. It provides methods to start, stop, reset, and retrieve the current time, with logic to manage paused and running states, which can be used to implement progress tracking features.

SongTimer.ino

File to implement the SongTimer class in the header file, providing the actual logic for tracking song playback time. It allows starting, stopping, resetting, and retrieving the elapsed time of a song. It uses the Arduino's millis() function to accurately track time.

SpotifyControllerTests.h

Header file for testing our program. It defines data structures and functions to test state transitions and synchronization between inputs, variables, and system states. It includes structures for representing input states and variable states, along with functions to test transitions and validate outcomes.

SpotifyControllerTests.ino

File containing our test cases to validate the state transitions and behavior of our system's FSM. It includes test data for input states, variable states, and expected outcomes. It also provides a function to evaluate individual transitions as well as a batch test to verify all cases.

SongTimerTests.ino

Contains unit tests for the SongTimer class.

LCDTests.ino

Contains unit tests for the SpotifyUtils class

Requirements:

1. Use PWM, ADC, or DAC
 - a. File: SpotifyController.ino
Line: 71
We use ADC. The analogRead() function reads the analog value from the pin and converts it to a digital value between 0 and 1023. This is our use of ADC to read potentiometer positions for the volume.
2. Have a watchdog timer
 - a. File: SpotifyController.ino
Lines: 38 - 50
This section initializes the watchdog timer with a specified interval. It checks for valid initialization and immediately refreshes the timer.

Lines: 54
In the loop function, the watchdog timer is periodically refreshed or pet, which resets the timer's countdown. If the refresh function is not called within the specific interval, the microcontroller will reset. The purpose of

this watchdog timer is to automatically reset the microcontroller if the code gets stuck or fails to periodically refresh the timer. In this case, if the code stops executing normally due to a hang or infinite loop, the microcontroller will automatically reset.

3. Have at least one interrupt service routine

a. File: SpotifyController.ino

Lines: 37, 128-134

Triggered by the play/pause button interrupt.

Lines: 38, 136-142

Triggered by the skip button interrupt. Both ISRs are set up using `attachInterrupt()` with the `FALLING` trigger mode, which means the interrupt is triggered when the pin transitions from HIGH to LOW. They include debouncing logic.

4. Serial communication

a. File: audio.py

Lines: 5, 23

Sets up the serial connection and reads from the serial port in the main loop. The entire script is built around a continuous loop that reads data from the Arduino's serial connection, processes the incoming data (converting a potentiometer value to system volume), and handles any potential errors during serial communication.

b. File: io_utils.ino

Lines: 3-8, 10-21

Reads incoming serial data character by character and stores it in the buffer. It also deserializes the JSON data received via serial communication and handles potential deserialization errors by printing them to the serial monitor.

c. File: main.py

Lines: 69-88, 91-104, 116-123, 125-127, 283-319

This code facilitates serial communication between a computer and an Arduino device, enabling bidirectional interaction for commands like pause, play, skip, and retrieving song duration.

d. File: SpotifyController.h

Line: 26

The serial buffer is declared to store data received via serial communication.

e. File: SpotifyController.ino

Lines: 25-28, 59-61, serial.print lines

The code uses serial communication to initialize the connection, check for incoming data, and process it. It frequently sends messages to the connected device, including commands like play, pause, skip, and getSongDuration.

11. How To Run Unit Tests

1. Set up the environment: connect Arduino to laptop, define the TEST macro in SpotifyController.h, and upload the compiled code for SpotifyController.ino to the Arduino. Then create a Python venv and use ``pip3 install -r requirements.txt``.
2. Go to Spotify and start the audio player on a Mac. (Note, we didn't implement volume changing on a Windows device.)
3. Go to the listed link on the How to Run section in the README, and follow the instructions there to get the access token. Once that is acquired, paste the token into the refreshToken field in ``main.py``.
4. Comment out lines 338-340 in ``main.py``.
5. Start up the Python server: run ``python3 main.py``. This will prompt you to enter the port the target Arduino is connected to, along with a list of devices that the computer is connected to.
6. Move back to the Arduino. Press the reset button to activate the code, at which point testing will begin on both the front and back end side.

12. Reflection on Goals

After reviewing our milestone progress, we successfully achieved end-to-end functionality with the Spotify API, allowing playback control using physical buttons. The play/pause button and the skip button work seamlessly, and the current song name is displayed on the LCD.

We also successfully adapted to new challenges to ensure our goals were met. Initially, connecting the Arduino to Wi-Fi was problematic, so we shifted to serial communication, which streamlined interactions between components. Another challenge arose with threading conflicts, where restarting the song and displaying the song name simultaneously caused issues. We resolved this by refining our thread management to ensure both tasks could operate without interference.

After incorporating the peer feedback from the milestone demos, we updated our FSM to address several key challenges and to reduce confusion. We simplified our FSM by removing the skipping state and integrating its functionality into the playing and paused states. The transitions now explicitly handle user actions like skipping while paused or playing, ensuring clear and predictable behavior. Guards were also added and clarified for all state transitions, and we made sure that they only occur under specific conditions. We also addressed edge cases such as API failures by displaying error messages in the console.

Our testing goals were partially met, as we outlined test cases and implemented FSM tests inspired by Lab 6. We created a set of FSM tests designed to validate the transitions between playback states under different input conditions. These tests simulate scenarios such as loading a song, pressing play or skip, and handling song completion. Each test is configured with predefined initial states, input conditions, and expected outputs to verify whether the FSM transitions and state variables behave as expected. However, we faced challenges that prevented full execution because of library compatibility issues. Moving forward, we plan to resolve those errors, and leverage other mocking tools to isolate dependencies for our tests.

Currently, we are working on improving the functionality of the potentiometer for volume control. While it adjusts the volume, it writes excessively to the buffer, which interferes with the responsiveness of the play/pause button. We're optimizing the process to make sure all controls operate without conflict. Additionally, we are developing a more seamless method to retrieve the Spotify access token. The current method requires manual intervention on our end, and we aim to streamline this process for greater efficiency and readability in maintaining API connectivity during playback sessions.

13. Review Spreadsheets

Defect	Fixed?
How do we go from skipping to paused, what would need to occur for this to happen, wouldn't we just always go back to the playing state to play the next song? Or is this supposed to mean that if the current song is paused then the skip button is pressed the next song will not play until the paused button is pressed again?	Fixed. We removed the skip state and simplified the logic by directly handling the transition. If the current song is paused and the skip button is pressed, the system now makes sure that the next song remains paused until the play button is pressed again. This is handled in the transition 2-3 (a) where the skip button is pressed while paused.
You have a skip variable that starts the next song but no where do you define stopping the current song before starting the next song	Fixed. The current song is "stopped" when skipped by resetting the song timer and sending a skip command to the Spotify API. This takes care of the current playback state and makes sure that it transitions smoothly to the next song without overlapping.
What happens in wait_for_song? Is this only at the beginning of the program or should there be a transition from state 4 to state 1 so that there is a slight delay between skipping songs? Should there be a transition from 3-1 to represent changing between songs once the current song has finished?	Not fixed. The wait_for_song state is designed to occur only at the beginning of the program because it waits for song information to be received from the laptop via the Spotify API before transitioning. We did not use this state again to represent a delay between skipping or changing songs because it transitions directly.
Where do you handle the case where one song finishes playing? Is this part of the 3-4 transition, if not it should be, or as mentioned above there could be a 3-1 transition for this.	Fixed. This case is currently handled with our revised transition 3-3 (b). When a song finishes, the system checks if the local song timer indicates the song's duration has elapsed, then it resets to prepare for the next song. Again, there is no explicit transition from playing to waiting for the song because 3-3 (b) makes sure the system fetches the next song and continues playing.
No inputs, outputs or guards	Fixed. We defined these elements in the new FSM and made sure they are properly associated with transitions and states.

Defect	Fixed?
What if there is no song queued? Unclear handling of the transition from wait_for_song to playing when a song is not queued or fails to load. I would suggest adding a separate state or error message for when no song is available in the queue.	Fixed. The system relies on Spotify's functionality, which ensures that a song is always queued when the API is queried. This removes the need for explicitly handling a "no song queued" scenario. Errors are handled externally by the Python script and are printed in the terminal. A future fix could be displaying the error message on the LCD.
Need more edge cases, like what happens when a play button is pressed repeatedly? Is there some kind of debouncing for when you guys have unintended rapid toggling? What happens if the song ends while it is paused, should it remain paused or should it transition to the next song?	Fixed. The system toggles between play and pause when the play button is pressed repeatedly. A debounce mechanism is implemented using a time threshold to make sure only one toggle is registered within a specific interval. Second, if the song ends while paused, the system transitions to the next song by fetching its information but does not start playback. This behavior assumes the system respects the paused state and does not automatically start the next song.
Doesn't describe how to handle API errors or Bluetooth disconnections.	Fixed. We do not use Bluetooth. API errors are managed by the Python script that interfaces with the Spotify API and logs it in the console. We could add an error state in the Arduino to handle API failures and display the error message.
The transition from skipping to wait_for_song is a little unclear, especially if there is no song queued or if the API doesn't respond. Maybe you could add an extra check to see whether a new song is available before transition out of the skipping state?	Fixed. We removed the skip state to simplify transitions and now it's handled in the playing and paused states. The system assumes the Spotify API always provides a new song when the skip command is sent.

Defect	Fixed?
1. Missing guards clarity to transition. 2. Missing actions to set outputs and variables.	Fixed. We explicitly defined these elements in the updated FSM and made sure they are properly associated with transitions and states.
Missing transition to enter from other states (may be after skipping?)	Fixed. We addressed this by removing the skip state from the FSM and integrating the skipping functionality into the existing playing and paused states.
Missing transition back to state 1 (wait_for_song) if song ends without skip button press	Fixed. If the song ends without a skip button press, the Spotify API always provides a new song when the skip command is sent.
Unclear in transition condition when moving to playing directly after skip button. May be split into two states or back to wait for song.	Fixed. We integrate skipping into the playing and paused states. For example, the system transitions to playing after skipping while paused. When it's playing and it's skipped, the song will continue playing using the Spotify API mechanism.

Defect	Fixed?
Having the state be skipping? (with a question mark) might not need the question mark just because it is conditional on a button press. As it stands now it seems like you are unsure on the naming of the state or its order by leaving the question mark (this is a small complaint :))	Fixed. We addressed this by removing the skip state from the FSM and integrating the skipping functionality into the existing playing and paused states.
I don't see any checking against a guard	Fixed. We explicitly defined these elements in the updated FSM and made sure they are properly associated with transitions and states.